VERIQ Verification API — Guide for the Flutter App

This guide covers how the official VERIQ mobile app reads a product label and verifies it against the VERIQ API. It describes the proof-of-concept API in this repository. Expect changes before production; see Production notes.

---

1. What is in the QR code

Every label encodes a short **plain-text** payload. It is not a URL:

```
VQ1.<token>.<sig>
```

Part	Format	Meaning
`VQ1`	literal	Payload format version 1
`<token>`	22 chars, base64url (`A–Z a–z 0–9 - _`)	128-bit secret label token
`<sig>`	16 chars, base64url	First 16 chars of `base64url(HMAC-SHA256(QR_SIGNING_SECRET, "VQ1." + token))`

Example: `VQ1.s1yQm3k0Xr7bT9cVwLqKQA.7RDp2xM_fK0aZ1nQ`

Regex for a quick client-side sanity check:

```
^VQ1\.[A-Za-z0-9_-]{22}\.[A-Za-z0-9_-]{16}$
```

The app must not try to validate the signature. The signing secret exists only on the server. Never ship it in the app. Send the payload to the API exactly as decoded (trimming surrounding whitespace is fine).

Because the payload is not a link, ordinary camera apps only show text and offer nothing to open. Older labels that encoded `https://…/v/<token>` are retired. The API answers them with `invalid` / `LEGACY_FORMAT`.

---

2. Calling the API

```
POST {API_BASE_URL}/api/verify
Content-Type: application/json
X-App-Key: <flutter-app key>
```

Headers

Header	Required	Notes
`Content-Type`	yes	`application/json`
`X-App-Key`	yes	Key issued to the Flutter app. The server's `APP_KEYS` contains `flutter-app:<key>`. Missing or unknown keys return **401**. The server records the client ID (`flutter-app`) on every scan.

Body

Field	Type	Required	Notes
`payload`	string (≤ 512)	yes	Raw QR content, e.g. `VQ1.<token>.<sig>`
`scannerId`	string (≤ 64)	recommended	Device or scanner identifier, e.g. `SCN-BER-01` or a per-install ID. Used by the repeat-scan rules.
`location`	string (≤ 80)	optional	Human-readable location, e.g. `Berlin, DE`. Used by the location rule. Collect GPS only with user consent.
`source`	`"camera"` | `"manual"`	optional	How the payload was captured. Default `manual`.

Status codes

HTTP	When
`200`	The request was processed. **Check `result`.** Invalid, forged and revoked codes are also `200`.
`400`	Body is not valid JSON or fails validation (e.g. missing `payload`)
`401`	`X-App-Key` missing (`APP_KEY_MISSING`) or not recognised (`APP_KEY_INVALID`). Nothing is logged.
`429`	Rate limit exceeded (per IP, default 60 requests/minute). Retry after the `RateLimit-Reset` header.
`5xx`	Server error. Show a "try again" state; do not cache.

`result` values

`result`	`reasonCode`	Show to the user	`product`
`authentic`	`FIRST_SCAN`, `REPEAT_SCAN`	Registered label (green)	present

`result`	`reasonCode`	Show to the user	`product`
`authentic`	`PRODUCT_RECALLED`, `PRODUCT_INACTIVE`	Registered, with a notice	present
`suspicious`	`SUSPICIOUS_PATTERN`	Registered, but the scan pattern is unusual. Possible copy (amber).	present
`invalid`	`BAD_SIGNATURE`	Not verified: modified or forged code (red)	`null`
`invalid`	`MALFORMED_TOKEN`	Not a VERIQ code	`null`
`invalid`	`LEGACY_FORMAT`	Retired label format	`null`
`invalid`	`TOKEN_NOT_FOUND`	Not registered	`null`
`revoked`	`TOKEN_REVOKED`	Label revoked (red)	`null`

Use `message` as a ready-made English explanation, or map `reasonCode` to localized strings. A `suspicious` result is a risk signal, not proof of counterfeiting. Word it carefully in the UI.

---

3. Example requests and responses

Authentic (first scan)

```
POST /api/verify
Content-Type: application/json
X-App-Key: <flutter-app key>

{ "payload": "VQ1.s1yQm3k0Xr7bT9cVwLqKQA.7RDp2xM_fK0aZ1nQ", "scannerId": "SCN-BER-01",
"location": "Berlin, DE", "source": "camera" }
```


```
        {
          "eventId": "evt_kFq238jN7bSN",
          "checkedAt": "2026-10-22T18:01:41.553Z",
          "result": "authentic",
          "reasonCode": "FIRST_SCAN",
          "message": "Registered label. This is the first verification of this cod
e.",
          "product": {
            "publicId": "VQ-E25A-BV7J",
            "name": "Meridian Chronograph 42 mm",
            "manufacturer": "Aurelle Horlogerie (fictional)",
            "category": "Watches",
            "batchNumber": "B-2026-0917",
            "productionDate": "2026-09-17",
            "status": "active"
```

```
          },
          "scan": {
            "scanNumber": 1,
            "repeatScan": false,
            "firstVerifiedAt": "2026-10-22T18:01:41.553Z",
            "flagged": false,
            "rulesTriggered": [],
            "rules": [
              { "code": "HIGH_SCAN_COUNT", "triggered": false, "observed": 1, "thresh
old": 4, "description": "…" },
              { "code": "MULTIPLE_DEVICES", "triggered": false, "observed": 1, "thres
hold": 3, "description": "…" },
              { "code": "MULTIPLE_LOCATIONS", "triggered": false, "observed": 1, "thr
eshold": 2, "description": "…" },
              { "code": "RAPID_DEVICE_SWITCH", "triggered": false, "observed": 0, "th
reshold": 30, "description": "…" }
            ]
          }
        }
```


Suspicious (same label seen on another scanner minutes later)


```
        {
          "eventId": "evt_TpWyaOoPjhQr",
          "checkedAt": "2026-10-22T18:04:57.799Z",
          "result": "suspicious",
          "reasonCode": "SUSPICIOUS_PATTERN",
          "message": "Registered label, but the scan pattern matches one or more anom
aly rules. The label may have been copied. The server cannot tell which physical label
is the original.",
          "product": { "publicId": "VQ-E25A-BV7J", "name": "Meridian Chronograph 42 m
m", "…": "…" },
          "scan": {
            "scanNumber": 2,
            "repeatScan": true,
            "firstVerifiedAt": "2026-10-22T18:04:54.984Z",
            "flagged": true,
            "rulesTriggered": ["RAPID_DEVICE_SWITCH"],
            "rules": [{ "code": "RAPID_DEVICE_SWITCH", "triggered": true, "observed":
0, "threshold": 30, "description": "…" }]
          }
        }
```


Invalid: malformed or unknown code

```
        {
          "eventId": "evt_3b9ZkQe0Wm1c",
          "checkedAt": "2026-10-22T18:06:10.120Z",
          "result": "invalid",
          "reasonCode": "MALFORMED_TOKEN",
          "message": "The scanned code is not a valid VERIQ verification code. It may
be damaged, modified or not issued by VERIQ.",
          "product": null,
          "scan": null
        }
```

Invalid: `BAD_SIGNATURE` (modified or forged payload)

```
        {
          "eventId": "evt_QBIPLYxDpJ16",
          "checkedAt": "2026-10-22T18:59:35.965Z",
          "result": "invalid",
          "reasonCode": "BAD_SIGNATURE",
          "message": "The code failed the signature check. It has been modified or fo
rged and was not issued by VERIQ.",
          "product": null,
          "scan": null
        }
```

401: missing or unknown app key

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json

{
  "error": {
    "code": "APP_KEY_MISSING",
    "message": "Verification is only available through an authorized VERIQ applicatio
n. The X-App-Key header is missing."
  }
}
```

`APP_KEY_INVALID` has the same shape. Treat any 401 from `/api/verify` as a configuration error ("This app version is not authorized. Please update."), not as a verification result.

---

4. Recommended Flutter packages

Package	Purpose
`mobile_scanner`	Camera QR scanning (ML Kit on Android, Vision on iOS)
`http`	HTTP client for `POST /api/verify`
`flutter_secure_storage` *(optional)*	Persist a per-install scanner ID
`geolocator` *(optional)*	Location, only with explicit consent

Minimal example

```
        import 'dart:convert';
        import 'package:flutter/material.dart';
        import 'package:http/http.dart' as http;
        import 'package:mobile_scanner/mobile_scanner.dart';

        const apiBase = String.fromEnvironment('VERIQ_API_BASE');   // e.g. https://api.veriq.example
        const appKey  = String.fromEnvironment('VERIQ_APP_KEY');    // flutter-app key (see Production notes)
        final payloadPattern = RegExp(r'^VQ1\.[A-Za-z0-9_-]{22}\.[A-Za-z0-9_-]{16}$');

        class VerifyResult {
          VerifyResult(this.result, this.reasonCode, this.message, this.product);
          final String result, reasonCode, message;
          final Map<String, dynamic>? product;
        }

        class UnauthorizedAppException implements Exception {
          UnauthorizedAppException(this.code);
          final String code;
        }

        Future<VerifyResult> verify(String payload, {required String scannerId, String? location}) async {
          final res = await http
              .post(
                Uri.parse('$apiBase/api/verify'),
                headers: {'Content-Type': 'application/json', 'X-App-Key': appKey},
                body: jsonEncode({'payload': payload.trim(), 'scannerId': scannerId, 'location': location, 'source': 'camera'}),
              )
              .timeout(const Duration(seconds: 10));

          final body = jsonDecode(res.body) as Map<String, dynamic>;
          if (res.statusCode == 401) throw UnauthorizedAppException(body['error']['co
```

```
de'] as String);
          if (res.statusCode != 200) throw Exception(body['error']?['message'] ?? 'Ve
rification failed (${res.statusCode})');
          return VerifyResult(body['result'], body['reasonCode'], body['message'], bo
dy['product']);
        }

        class ScanScreen extends StatefulWidget {
          const ScanScreen({super.key, required this.scannerId});
          final String scannerId;
          @override
          State<ScanScreen> createState() => _ScanScreenState();
        }

        class _ScanScreenState extends State<ScanScreen> {
          final controller = MobileScannerController(formats: [BarcodeFormat.qrCod
e]);
          bool busy = false;

          Future<void> onDetect(BarcodeCapture capture) async {
            final raw = capture.barcodes.firstOrNull?.rawValue;
            if (busy || raw == null) return;
            busy = true;
            await controller.stop();
            try {
              // Non-VERIQ codes can be rejected locally; the server still decides va
lidity of VQ1 codes.
              if (!payloadPattern.hasMatch(raw.trim())) {
                _show('Not a VERIQ label');
              } else {
                final r = await verify(raw, scannerId: widget.scannerId);
                _show('${r.result.toUpperCase()}: ${r.message}');
              }
            } on UnauthorizedAppException {
              _show('This app is not authorized to verify labels. Please update the a
pp.');
            } catch (e) {
              _show('Could not verify: $e');
            } finally {
              busy = false;
            }
          }

          void _show(String text) => showDialog(context: context, builder: (_) => Ale
rtDialog(content: Text(text)))
              .then((_) => controller.start());
```

```
          @override
          Widget build(BuildContext context) => Scaffold(body: MobileScanner(controll
er: controller, onDetect: onDetect));
        }
```

Notes: - Do not log or persist raw payloads. They are bearer credentials for that label. - Treat each scan as one call. Do not auto-retry on `200` results; retry only on timeouts and 5xx, with backoff. - `scannerId` should be stable per device or install so the repeat-scan rules work.

Production notes

- **An app key is an identifier, not a secret.** Anything compiled into an app can be extracted. Combine the key with **app attestation**: Google Play Integrity on Android and App Attest / DeviceCheck on iOS. The server should verify the attestation token before accepting a scan, and keys should be rotatable per app version.
- Use HTTPS only, with certificate pinning in the app.
- Scanner and device identities should come from device enrolment, not self-reported strings.
- Copying the label is still possible: the signature prevents forging *new* payloads, not photographing existing ones. The anomaly rules flag suspicious patterns; physical protection (tamper-evident or secure labels, secure NFC) is needed for stronger guarantees.